Block filesystems



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Block devices



Block vs. raw flash

- Storage devices are classified in two main types: **block devices** and **raw flash devices**
  - They are handled by different subsystems and different filesystems
- **Block devices** can be read and written to on a per-block basis, in random order, without erasing.
  - Hard disks, RAM disks
  - USB keys, SSD, SD cards, eMMC: these are based on flash storage, but have an integrated controller that emulates a block device, managing the flash in a transparent way.
- **Raw flash devices** are driven by a controller on the SoC. They can be read, but writing requires prior erasing, and often occurs on a larger size than the "block" size.
  - NOR flash, NAND flash



Block device list

- The list of all block devices available in the system can be found in `/proc/partitions`

```
$ cat /proc/partitions
major   minor    #blocks name

 179        0    3866624 mmcblk0
 179        1      73712 mmcblk0p1
 179        2    3792896 mmcblk0p2
   8        0  976762584 sda
   8        1    1060258 sda1
   8        2  975699742 sda2
```

- `/sys/block/` also stores information about each block device, for example whether it is removable storage or not.



Partitioning

- Block devices can be partitioned to store different parts of a system
- The partition table is stored inside the device itself, and is read and analyzed automatically by the Linux kernel
  - `mmcblk0` is the entire device
  - `mmcblk0p2` is the second partition of `mmcblk0`
- Two partition table formats:
  - *MBR*, the legacy format
  - *GPT*, the new format, now used by all modern operating systems, supporting disks bigger than 2 TB.
- Numerous tools to create and modify the partitions on a block device: `fdisk`, `cfdisk`, `sfdisk`, `parted`, etc.



Transferring data to a block device

- It is often necessary to transfer data to or from a block device in a *raw* way
  - Especially to write a *filesystem image* to a block device
- This directly writes to the block device itself, bypassing any filesystem layer.
- The block devices in `/dev/` allow such *raw* access
- `dd` is the tool of choice for such transfers:
  - `dd if=/dev/mmcblk0p1 of=testfile bs=1M count=16`
    Transfers 16 blocks of 1 MB from `/dev/mmcblk0p1` to `testfile`
  - `dd if=testfile of=/dev/sda2 bs=1M seek=4`
    Transfers the complete contents of `testfile` to `/dev/sda2`, by blocks of 1 MB, but starting at offset 4 MB in `/dev/sda2`
  - **Typical mistake**: copying a file (which is not a filesystem image) to a filesystem without mounting it first:
    ```
    dd if=zImage of=/dev/sde1
    ```
    Instead, you should use:
    ```
    sudo mount /dev/sde1 /boot
    cp zImage /boot/
    ```



Available block filesystems



Ext2

One of the earliest Linux filesystem, introduced in 1993

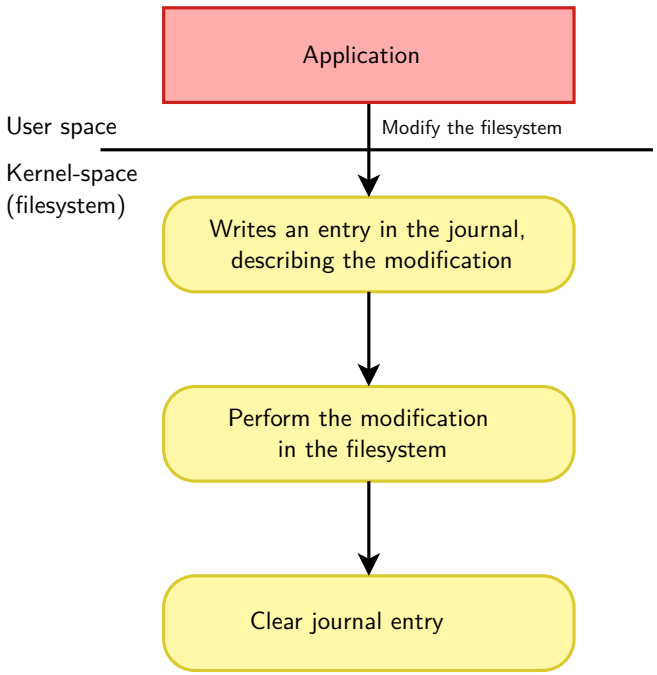
- filesystems/ext2
- Still actively supported. Low metadata overhead, module size and RAM usage
- But risk of metadata corruption after an unclean shutdown. You then need to run `e2fsck`, which takes time and may need operator intervention. Can't reboot autonomously.
- First successor: `ext3` (2001), addressing this limitation with *Journaling* (see next slides) but wasn't scaling well. Now deprecated.
- It supports all features Linux needs in a root filesystem: permissions, ownership, device files, symbolic links, etc.
- Date range: December 14, 1901 – January 18, 2038, because of 32 bit dates!

Not recommended for embedded systems today!

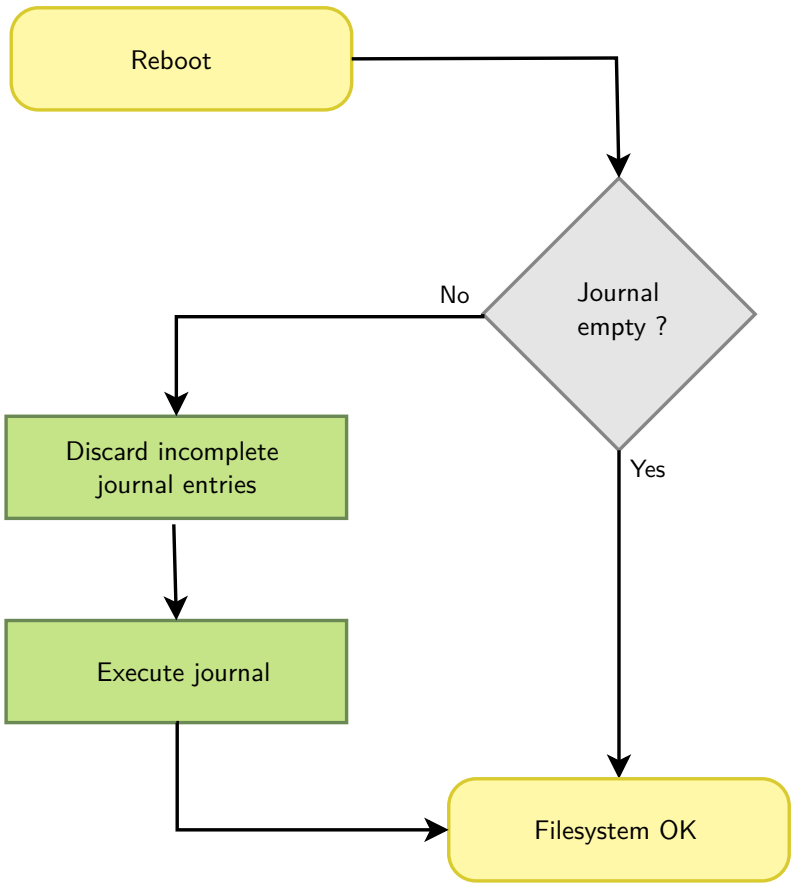


Journaled filesystems

- Unlike simpler filesystems (`ext2`, `vfat`...), designed to stay in a coherent state even after system crashes or a sudden poweroff.
- Writes are first described in the journal before being committed to files (can be all writes, or only metadata writes depending on the configuration)



Filesystem recovery after crashes



- Thanks to the journal, the recovery at boot time is quick, since the operations in progress at the moment of the unclean shutdown are clearly identified. There's no need for a full filesystem check.
- Does not mean that the latest writes made it to the storage: this depends on syncing the changes to the filesystem.

See https://en.wikipedia.org/wiki/Journaling_file_system for further details.



Ext4

The modern successor of Ext2

- First introduced in 2006, filesystem with Journaling, without `ext3` limitations.
- Still actively developed (new features added). However, considered in 2008 by Ted Ts'o as a "stop-gap" based on old technologies.
- The default filesystem choice for many GNU/Linux distributions (Debian, Ubuntu)
- The `ext4` driver also supports `ext2` and `ext3` (one driver is sufficient).
- Noteworthy feature: transparent encryption (but compression not available).
- Minimum partition size to have a journal: 2MiB (256 inodes).
- Minimum partition size without a journal: 64KiB (only 16 inodes!).

https://en.wikipedia.org/wiki/Ext4



XFS

A Journaling filesystem

- Since 1994 (started by Silicon Graphics for the IRIX OS)
- Actively maintained and developed by Red Hat now
- Features: variable block size, direct I/O, online growth...
- Minimum partition size: 16MiB (9.7MiB of free space)

https://en.wikipedia.org/wiki/XFS



Btrfs

A copy-on-write filesystem

- Pronounced as “better F S”, “butter F S” or “b-tree F S”, since 2009.
- A modern filesystem with many advanced features: volumes, snapshots, transparent compression... Looks great for storage experts.
- Minimum partition size: 109MiB (only 32MiB of free space).
- However, big module size and long initialization time (bad for boot time)

https://en.wikipedia.org/wiki/Btrfs



F2FS — Flash-Friendly File System

A log-structured filesystem

- Since 2012 (started by Samsung, actively maintained)
- Designed from the start to take into account the characteristics of solid-state based storage (eMMC, SD, SSD)
- In particular, trying to make most writes sequential (best on SSD)
- Support for transparent encryption and compression (LZO, LZ4, Zstd), possible on a file by file (or file type) basis, through extended file attributes.
- Maximum partition size: 16TB, maximum file size: 3.94TB
- Minimum partition size: 52MiB (8MiB free space)

https://en.wikipedia.org/wiki/F2FS



SquashFS — A Read-Only and Compressed File System

The most popular choice for this usage

- Started by Phillip Lougher, since 2009 in the mainline kernel, actively maintained.
- Fine for parts of a filesystem which can be read-only (kernel, binaries...)
- Used in most live CDs and live USB distributions
- Supports several compression algorithms (Gzip, LZO, XZ, LZ4, Zstd)
- Supposed to give priority to compression ratio vs read performance
- Suitable for very small partitions

https://en.wikipedia.org/wiki/SquashFS



EROFS — Enhanced Read-Only File System

A more recent read-only, compressed solution

- Started by Gao Xiang (Huawei), since 2019 in the mainline kernel.
- Used in particular in Android phones (Huawei, Xiaomi, Oppo...)
- Supposed to give priority to read performance vs compression ratio
- EROFS implements compression into fixed 4KB blocks (better for read performance), while SquashFS uses fixed-sized blocks of uncompressed data.
- Unlike Squashfs, EROFS also allows for random access to files in directories.
- Development seems more active than on SquashFS.
- Suitable for very small partitions

https://en.wikipedia.org/wiki/EROFS



Our advice for choosing the best filesystem

- Some filesystems will work better than others depending on how you use them.
- Fortunately, filesystems are easy to benchmark, being transparent to applications:
  - Format your storage with each filesystem
  - Copy your data to it
  - Run your system on it and benchmark its performance.
  - Keep the one working best in your case.
- If you haven't done benchmarks yet, a good default choice is `ext4` for read/write partitions.



Filesystem benchmarks

	Boot time [1]	Mount time	Read	Seq. read	Write	Seq. write	read write delete	Delete	Space
ext4	very good	very good	fair	good	best	very good	good	good	good
xfs	bad	average	fair	good	very good	best	average	fair	fair
btrfs	worst	good	fair	good	good	good	fair	worst	good
f2fs	fair	average	good	good	fair	very good	very good	average	worst
squashfs	excel-lent	best	very good	very good					best
erofs	best	best	best	best					very good

1. Boot time = Module loading time + Mount time

See our presentation for more details and benchmarks (Linux 6.3, ARM32 BeagleBone Black):



Compatibility filesystems

Linux also supports several other filesystem formats, mainly to be interoperable with other operating systems:

- `vfat` (CONFIG_VFAT_FS) for compatibility with the FAT filesystem used in the Windows world and on numerous removable devices
  - Also convenient to store bootloader binaries (FAT easy to understand for ROM code)
  - This filesystem does *not* support features like permissions, ownership, symbolic links, etc. Cannot be used for a Linux root filesystem.
  - Linux now supports the exFAT filesystem too (CONFIG_EXFAT_FS).
- `ntfs` (CONFIG_NTFS_FS) for compatibility with Windows NTFS filesystem.
- `hfs` (CONFIG_HFS_FS) for compatibility with the MacOS HFS filesystem.



tmpfs: filesystem in RAM

CONFIG_TMPFS

- Not a block filesystem of course!
- Perfect to store temporary data in RAM: system log files, connection data, temporary files...
- More space-efficient than ramdisks: files are directly in the file cache, grows and shrinks to accommodate stored files
- How to use: choose a name to distinguish the various tmpfs instances you have (unlike in most other filesystems, each tmpfs instance is different). Examples:

```
mount -t tmpfs run /run
mount -t tmpfs shm /dev/shm
```

- See filesystems/tmpfs in kernel documentation.



Using block filesystems



Creating filesystems

- To create an empty ext4 filesystem on a block device or inside an already-existing image file
  - `mkfs.ext4 /dev/sda3`
  - `mkfs.ext4 disk.img`
- To create a filesystem image from a directory containing all your files and directories
  - For some filesystems, there are utilities to create a filesystem image from an existing directory:
    - *ext2*: `genext2fs -d rootfs/ rootfs.img`
    - *squashfs*: `mksquashfs rootfs/ rootfs.sqfs` (details later)
    - *erofs*: `mkfs.erofs rootfs.erofs rootfs/`
  - For other (read-write) filesystems: create a disk image, format it, mount it (see next slides), copy contents and umount.
  - Your image is then ready to be transferred to your block device



Mounting filesystem images

- Once a filesystem image has been created, one can access and modify its contents from the development workstation, using the **loop** mechanism:
- Example:

```
mkdir /mnt/test
mount -t ext4 -o loop rootfs.img /mnt/test
```

- In the `/mnt/test` directory, one can access and modify the contents of the `rootfs.img` file.
- This is possible thanks to `loop`, which is a kernel driver that emulates a block device with the contents of a file.
- Note: `-o loop` no longer necessary with recent versions of `mount` from *GNU Coreutils*. Not true with BusyBox `mount`.
- Do not forget to run `umount` before using the filesystem image!



How to access partitions in a disk image

- You may have dumped a complete block device (with partitions) into a disk image.
- The `losetup` command allows to manually associate a loop device to a file, and offers a `--partscan` option allowing to also create extra block device files for the partitions inside the image:

```
$ sudo losetup -f --show --partscan disk.img
/dev/loop2

$ ls -la /dev/loop2*
brw-rw---- 1 root disk   7,  2 Jan 14 10:50 /dev/loop2
brw-rw---- 1 root disk 259, 11 Jan 14 10:50 /dev/loop2p1
brw-rw---- 1 root disk 259, 12 Jan 14 10:50 /dev/loop2p2
```

- Each partition can then be accessed individually, for example:

```
$ mount /dev/loop2p2 /mnt/rootfs
```



Creating squashfs filesystems

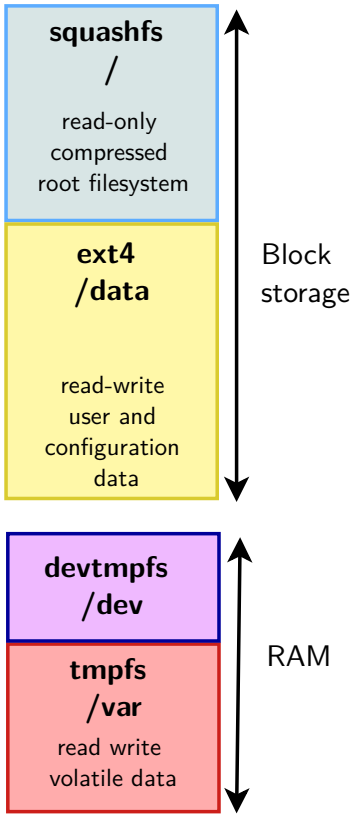
- Need to install the `squashfs-tools` package
- Can only create an image: creating an empty *squashfs* filesystem would be useless, since it's read-only.
- To create a *squashfs* image:
  - `mksquashfs data/ data.sqfs -noappend`
  - `-noappend`: re-create the image from scratch rather than appending to it
- Examples mounting a squashfs filesystem:
  - Same way as for other block filesystems
  - `mount -o loop data.sqfs /mnt` (filesystem image on the host)
  - `mount /dev/<device> /mnt` (on the target)
- Similar commands exist for EROFS



Mixing read-only and read-write filesystems

Good idea to split your block storage into:

- A compressed read-only partition (`SquashFS`) Typically used for the root filesystem (binaries, kernel...). Compression saves space. Read-only access protects your system from mistakes and data corruption.
- A read-write partition with a journaled filesystem (like `ext4`) Used to store user or configuration data. Journaling guarantees filesystem integrity after power off or crashes.
- Ram storage for temporary files (`tmpfs`)





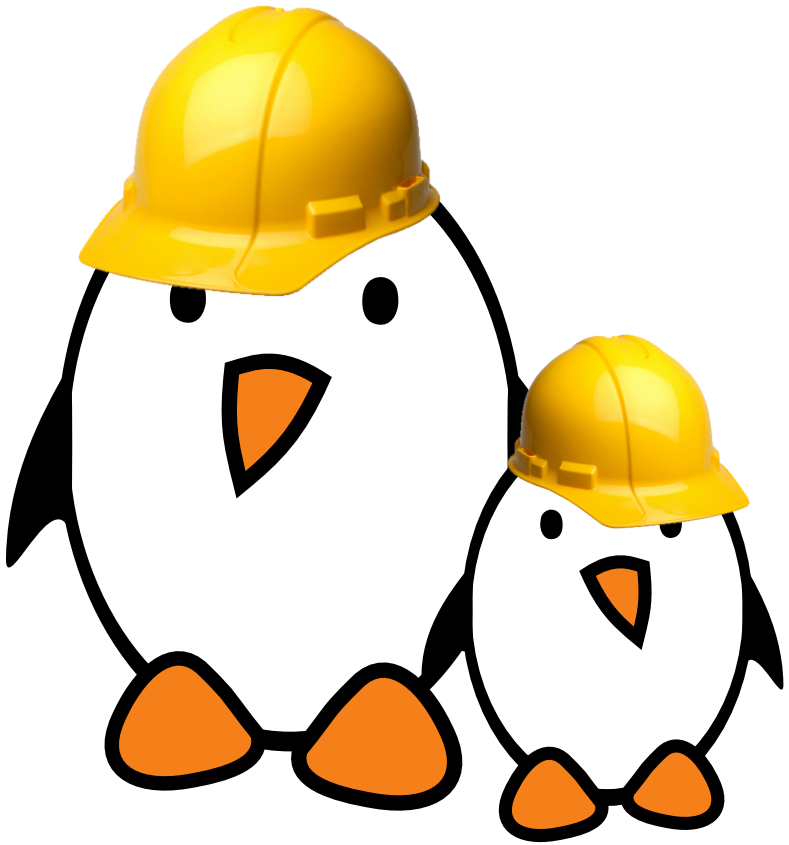
Issues with flash-based block storage

- Flash storage made available only through a block interface.
- Hence, no way to access a low level flash interface and use the Linux filesystems doing wear leveling.
- No details about the layer (Flash Translation Layer) they use. Details are kept as trade secrets, and may hide poor implementations.
- Not knowing about the wear leveling algorithm, it is highly recommended to limit the number of writes to these devices.
- Using industrial grade storage devices (MMC/SD, USB) is also recommended.

Note that some SD cards report their erase block size, available in `/sys/bus/mmc/devices/<dev>/preferred_erase_size`.

For more details, see *Optimizing Linux with cheap flash drives* from Arnd Bergmann.

Practical lab - Block filesystems



- Creating further partitions on your SD card
- Booting a system with a mix of filesystems: *SquashFS* for the root filesystem, *ext4* for data, and *tmpfs* for temporary system files.
- Loading everything from the SD card, including the kernel and device tree.